

1. **The Node.js runtime:** What Node.js is: JavaScript outside the browser · Running code: `node file.js`, the REPL and `--watch` · Why a framework, and why Express
2. **Modules:** Modules: `require` and `module.exports` · What `require` finds: built-in modules, packages and your files · ES modules: `import`, `export` and `"type": "module"`
3. **npm and package.json:** `package.json`: the project's manifest · Installing packages: dependencies, `devDependencies` and `node_modules` · Version numbers and ranges: what `^4.19.0` means · `package-lock.json`, `npm ci` and what to commit
4. **Layout and configuration:** A Node/Express project layout · Environment variables and `process.env` · `.env` files, config and keeping secrets out of Git
5. **Asynchronous code:** Blocking and non-blocking: why Node code is asynchronous · Callbacks and the error-first convention · Promises: a value that arrives later · `async` and `await` · Errors in async code: `try/catch` every `await` · Independent work in parallel: `Promise.all`
6. **The event loop:** The event loop: one thread, a call stack and queues · Microtasks before tasks: predicting the order · Never block the event loop

The Node.js runtime

What Node.js is: JavaScript outside the browser

Node.js is a runtime: a program that runs JavaScript outside the browser. It packages V8, the JavaScript engine of Chrome, with platform APIs for files, the network and processes. It has no page, so there is no document and no window.

Same language, two environments

	Browser	Node.js
Engine	V8 (Chrome, Edge), SpiderMonkey (Firefox)...	V8
What runs	Scripts loaded by a web page	Files you start with <code>node file.js</code>
The page	document, window, DOM events	None: document is not defined
Global object	window (also <code>globalThis</code>)	<code>globalThis</code> (no window)
Files and network	Only through the page (<code>fetch</code> , uploads); no access to your disk	<code>fs</code> (files), <code>http</code> (servers), sockets, processes
Program information	location, navigator	<code>process</code> : <code>process.env</code> , <code>process.argv</code> , <code>process.exit()</code>
console, timers, fetch	Yes	Yes (fetch since Node 18)

Example. Save `hello.js` with `console.log('Running on Node', process.version);` and run `node hello.js`: the terminal prints something like `Running on Node v24.11.0`. Add `console.log(typeof document);` and Node prints `undefined`; add `document.title = 'x';` and it stops with `ReferenceError: document is not defined`. The examples here target **Node 24 LTS**. An **LTS** (Long-Term Support) release gets fixes for about three years, which makes it the safe choice for anything you deploy; the newer "Current" release line is for trying new features.

Common mistake. Thinking Node is a new language or a framework. JavaScript is the language, Node is the **runtime** where it runs, and Express (see Routes and middleware) is a **framework** that runs on Node. A related slip: pasting browser code (`document.querySelector`, `alert`, `localStorage`) into a server file. A server has no page and no user sitting in front of it; it only receives requests and sends responses.

Running code: node file.js, the REPL and --watch

You run a file with `node file.js`, try single expressions in the REPL (type `node` on its own), and during development use `node --watch file.js`, which restarts the program every time you save.

- `node file.js`: run a file (`node file` works too: `.js` is assumed).
- `node`: the REPL; `.exit` to leave.
- `node --watch file.js`: restart on every save (development only).
- `process.argv`: the words typed on the command line, as strings; `process.argv[2]` is the first word after the file name.
- **Ctrl+C** stops the program running in that terminal.

```
// greet.js
const name = process.argv[2] || 'stranger';
console.log(`Hello, ${name}!`);

// In the terminal:
// $ node greet.js Ana
// Hello, Ana!
// $ node greet.js
// Hello, stranger!
// $ node
// > 2 ** 10
// 1024
// > .exit
```

Example. Say a project's npm start runs `node --watch src/server.js`. You change the `/health` route to answer `{ status: 'ok', time: Date.now() }` and save: the terminal shows that the process restarted, and the next `curl http://localhost:3000/health` already returns the new field. Started with plain `node`, the server would keep answering with the old code until you pressed `Ctrl+C` and started it again.

Common mistake. Editing the code, retrying the request and seeing no change, because the server was started with plain `node` and still runs the old version. Another one: starting the server again in a second terminal while the first is still running. The second fails with `Error: listen EADDRINUSE: address already in use :::3000: port 3000 is taken`. Stop the old process with `Ctrl+C` in its terminal, or start the new one on another port.

Why a framework, and why Express

Node gives you the runtime and a low-level `http` module; a **web framework** adds the conventions a real API needs: **routing, a middleware pipeline and a standard structure**. The examples on this site use **Express**, the most widely used one and the model for the others.

Four Node frameworks you will hear about

Framework	Style	Typical use
Express	Minimal, unopinionated, huge ecosystem of middleware	Small and medium APIs; the worked example on this site
Fastify	Very fast; validates requests against schemas; plugins	APIs where performance matters
Koa	Minimal; middleware written with <code>async/await</code>	Lightweight custom servers
NestJS	Structured, TypeScript first, many built-in conventions	Large applications built by big teams

```
// Plain Node: one function receives every request
const http = require('node:http');

const server = http.createServer((req, res) => {
  if (req.method === 'GET' && req.url === '/health') {
    res.writeHead(200, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify({ status: 'ok' }));
  } else {
    res.writeHead(404, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify({ error: 'Not found' }));
  }
});
server.listen(3000);

// The same route with Express:
// app.get('/health', (req, res) => res.json({ status: 'ok' }));
```

Example. Adding `GET /tasks/:id` to the plain `http` server means splitting `req.url` by hand, checking the method, converting the `id` to a number and repeating the `Content-Type` header in every answer. In Express it is one line, `app.get('/tasks/:id', getTask)`, and `req.params.id` already holds the `id`. Multiply that by twenty routes and the framework saves days.

Common mistake. "Express replaces Node." Express is a package that **runs inside** Node: it is installed with npm, loaded with `require('express')`, and uses the `http` module underneath. Everything in this section (modules, npm, `process.env`, the event loop) applies to every Express app.

Modules

Modules: `require` and `module.exports`

In Node every file is a **module** with its own private scope. A file shares a value by assigning it to `module.exports`; another file receives that value with `require('./path')`. This system is **CommonJS**, Node's original one and the one the server examples here use.

- Paths to your own files start with `./` (this folder) or `../` (the parent folder); the `.js` can be left out.
- `module.exports = value` replaces what the file exports; the last assignment wins.
- `exports` is a shortcut that starts as the same object: `exports.total = 1` works, but `exports = { total: 1 }` does **not** (it only re-points the shortcut).

```
// src/utils/money.js
const VAT = 0.21; // private: not exported
function withVat(price) {
  return Math.round(price * (1 + VAT) * 100) / 100;
}
module.exports = withVat; // export ONE function

// src/report.js
const withVat = require('./utils/money'); // ./ = relative to THIS file
console.log(withVat(100)); // 121
console.log(typeof VAT); // 'undefined': VAT stayed private
```

```
// How Node runs each file (simplified): your code is wrapped in a
// function that receives "module" and "exports".
function runModule(code) {
  const module = { exports: {} };
  code(module, module.exports);
  return module.exports; // what require() gives back
}

const good = runModule((module) => {
  module.exports = { greet: (name) => 'Hi ' + name };
});
console.log(good.greet('Ana')); // Hi Ana

const bad = runModule((module, exports) => {
  exports = { greet: (name) => 'Hi ' + name }; // re-points the shortcut only
});
console.log(bad); // {}: nothing was exported
```

Example. A small Express API can be five files connected only by `require` and `module.exports`: `server.js` requires `./app`; `app.js` requires `express`, `./middleware/logger` and `./routes/tasks`; the router requires `../controllers/tasksController`. Follow the arrows and you have the whole app. The router lives in `src/routes/`, so to reach `src/controllers/` it first goes up one folder with `../`.

Common mistake. Forgetting the export. If `logger.js` defines the function but never assigns `module.exports`, `require('./middleware/logger')` returns an empty object `{}`, and `app.use(logger)` throws `TypeError: app.use() requires a middleware function`. When a required value is `{}` or `undefined`, look at the **last lines** of the file you required.

What `require` finds: built-in modules, packages and your files

`require` reads its argument in three ways: `node:fs` is a **built-in module** that comes with Node; a bare name such as `express` is a **package** looked up in `node_modules/`; and a path starting with `./` or `../` is **one of your own files**.

Three kinds of `require`

You write	Node loads	Install first?
<code>require('node:fs')</code>	The built-in file-system module	No: part of Node
<code>require('express')</code>	<code>node_modules/express/</code> (searching up the folders)	Yes: <code>npm install express</code>
<code>require('./app')</code>	<code>app.js</code> (or <code>app/index.js</code>) in the same folder as this file	No: it is your file
<code>require('../controllers/tasksController')</code>	Up one folder, then <code>controllers/tasksController.js</code>	No

```
// What a controller module exports: an object with two functions
const tasksController = {
  listTasks() { return 'listing tasks'; },
  createTask() { return 'creating a task'; },
};

// Destructuring takes properties out by name
const { listTasks, createTask } = tasksController;
console.log(listTasks()); // listing tasks
console.log(createTask()); // creating a task

// A name that is not a property gives undefined
const { deleteTask } = tasksController;
console.log(deleteTask); // undefined
```

Example. A typical router file starts with `const express = require("express");` (a package from `node_modules`) and `const { listTasks, createTask } = require("../controllers/tasksController");` (your own file, one folder up). If you later add `deleteTask` to that destructuring but forget it in the controller's `module.exports = { ... }`, the variable is `undefined` and `router.delete('/:id', deleteTask)` throws at start-up: `Route.delete()` requires a callback function but got a `[object Undefined]`.

Common mistake. Writing `require('app')` for your own `app.js`. Without `./`, Node treats `app` as a **package** name, searches `node_modules`, and fails with `Cannot find module 'app'`, even though `app.js` is right next to the file. Paths to your own files always start with `./` or `../`.

ES modules: import, export and "type": "module"

ES modules (ESM) are the module syntax built into the JavaScript language, `import` and `export`; browsers use them and so do front-end tools such as Vite and React. Node supports both systems: a file is read as ESM when it ends in `.mjs` or when `package.json` says `"type": "module"`.

The same module in both systems

	CommonJS (server examples here)	ES modules (browsers, Vite, React)
Export one thing	<code>module.exports = app;</code>	<code>export default app;</code>
Export several	<code>module.exports = { a, b };</code>	<code>export function a() {}</code> Or <code>export { a, b };</code>
Import one thing	<code>const app = require('./app');</code>	<code>import app from './app.js';</code>
Import some	<code>const { a } = require('./lib');</code>	<code>import { a } from './lib.js';</code>
Chosen by	<code>.cjs</code> , or <code>.js</code> with <code>"type": "commonjs"</code>	<code>.mjs</code> , or <code>.js</code> with <code>"type": "module"</code>
Top-level await	No	Yes

Example. You paste `import express from 'express';` from a tutorial into an `app.js` whose other lines use `require`. With `"type": "commonjs"` in `package.json` (as `npm init -y` writes it), Node stops with `SyntaxError: Cannot use import statement outside a module`. With no `"type"` (as in many older projects' `package.json`), Node re-runs the file as an ES module and fails on the next line instead: `ReferenceError: require is not defined in ES module scope`. Either way the fix is the same: write the line in the project's style, `const express = require('express');`.

Common mistake. Adding `"type": "module"` to `package.json` to make one copied `import` line work. Every `.js` file of the project is then read as ESM, so every `require` and `module.exports` breaks at once. Pick one system per project; the server examples on this site use CommonJS.

npm and package.json

package.json: the project's manifest

package.json **describes a Node project: its name and version, the packages it depends on, and the scripts you run with npm. You create it with** `npm init -y`, **and it is always committed.**

The fields you will meet

Field	What it says	Example
name, version	The project's identity	"tasks-api", "1.0.0"
private	true: npm refuses to publish it to the public registry by accident	true
type	How .js files are read: CommonJS or ES modules	"commonjs"
main	The entry file	"src/server.js"
scripts	Commands for npm start, npm test, npm run <name>	"dev": "node --watch src/server.js"
dependencies	Packages the app needs to run	"express": "^4.22.3"
devDependencies	Packages needed only to develop and test	"jest": "^29.7.0"

JSON

```
{
  "name": "tasks-api",
  "version": "1.0.0",
  "private": true,
  "type": "commonjs",
  "main": "src/server.js",
  "scripts": {
    "start": "node src/server.js",
    "dev": "node --watch src/server.js",
    "test": "jest"
  },
  "dependencies": {
    "express": "^4.22.3"
  },
  "devDependencies": {
    "jest": "^29.7.0",
    "supertest": "^7.0.0"
  }
}
```

Example. Some projects have a single script, "start": "node --watch src/server.js", so npm start both starts the server and restarts it on every save. The more common split: start runs plain node src/server.js (what a production server runs) and dev adds --watch for development (npm run dev). Both work; what matters is that a teammate finds in one place how to start your project.

Common mistake. Typing npm dev instead of npm run dev. Only a few names (start, test, stop, restart) work without run; for any other, npm answers Unknown command: "dev". Another classic: adding a comment such as // my scripts inside package.json. JSON has no comments, so npm stops with a parse error.

Installing packages: dependencies, devDependencies and node_modules

npm install express **downloads a package into the node_modules/ folder and records it in package.json: under dependencies if the app needs it to run, or under devDependencies (npm install --save-dev jest) if it is only for developing and testing.**

The npm commands you will use most

Command	What it does
npm install	Installs everything listed in package.json (both lists) into node_modules/
npm install express@4	Adds Express 4 to dependencies and installs it (@4 picks the major version)
npm install --save-dev jest	Adds jest to devDependencies (short form: npm i -D jest)
npm uninstall express	Removes it from node_modules/ and from package.json
npm ci	Clean install that follows package-lock.json exactly (see "package-lock.json")

Command	What it does
<code>npm ls</code>	Shows which packages are installed

Example. Setting up a new Express 4 API: `npm init -y`, then `npm install express@4`. Three things appear: an "express": "^4.x.x" line under dependencies, a `package-lock.json` file, and a `node_modules/` folder with Express and its own dependencies. The `@4` matters: this project wants Express 4, but a plain `npm install express` today installs **Express 5**, the current major version. Later, for automated tests (see Designing APIs), `npm install --save-dev jest supertest` adds two devDependencies.

Common mistake. Committing `node_modules` to GitHub: thousands of files of other people's code, some built specifically for **your** operating system and broken on a teammate's. Create a `.gitignore` containing `node_modules/` **before** the first commit. If it is already committed, `git rm -r --cached node_modules` removes it from the repository while keeping it on your disk.

Version numbers and ranges: what ^4.19.0 means

Packages use **semantic versioning**: MAJOR.MINOR.PATCH. A **caret range** such as `^4.19.0` accepts any later `4.x.y` version but never `5.0.0`; a **tilde range** `~4.19.0` accepts only `4.19.x`.

What each range lets npm install

Range	Accepts	Rejects
<code>^4.19.0</code>	4.19.0 up to the newest 4.x.y	4.18.2, 5.0.0
<code>~4.19.0</code>	4.19.0 up to the newest 4.19.x	4.20.0
<code>4.19.0</code>	only 4.19.0	everything else
<code>^0.4.2</code>	0.4.2 up to the newest 0.4.x	0.5.0
<code>*</code>	any version at all	nothing: avoid it

```
// A simplified version of npm's caret rule, for versions >= 1.0.0
function caretAccepts(range, version) {
  const r = range.replace('^', '').split('.').map(Number);
  const v = version.split('.').map(Number);
  if (v[0] !== r[0]) return false; // another MAJOR may break your code
  if (v[1] !== r[1]) return v[1] > r[1]; // a newer MINOR is fine
  return v[2] >= r[2]; // same MINOR: PATCH must not be older
}

console.log(caretAccepts('^4.19.0', '4.22.3')); // true
console.log(caretAccepts('^4.19.0', '4.18.9')); // false: older
console.log(caretAccepts('^4.19.0', '5.0.0')); // false: new major
```

Example. A project's `package.json` once said "express": "^4.19.2" and now says "^4.22.3". A developer who installed with the first range in September and a teammate who installs today can get different 4.x versions, both allowed by the caret. That is compatible, but not identical, which is exactly the problem the **lockfile** solves: it records the precise version that was installed, so everyone gets the same one (next card).

Common mistake. Reading `^4.19.0` as "version 4.19.0". It means "4.19.0 or any newer 4.x". The opposite mistake is changing the major by hand ("`^5.0.0`") "to update". A new major version may change behaviour your code relies on, so read the package's migration guide first.

package-lock.json, npm ci and what to commit

`package-lock.json` **records the exact version of every installed package, transitive ones included, so every machine installs the same set. Commit it together with `package.json`, and use `npm ci` to install exactly what it lists.**

What goes into the repository

File or folder	Commit it?	Why
<code>package.json</code>	Yes	You write it: dependencies and scripts
<code>package-lock.json</code>	Yes	Generated, but it pins the exact versions everyone must get
<code>node_modules/</code>	No : list it in <code>.gitignore</code>	Rebuilt by <code>npm ci</code> or <code>npm install</code> ; huge and partly OS-specific
<code>.env</code>	No : list it in <code>.gitignore</code>	Holds secrets (see ".env files")

File or folder	Commit it?	Why
.env.example	Yes	Shows which variables are needed, with placeholder values

```
# .gitignore for a Node project
node_modules/
.env
npm-debug.log*
.DS_Store
```

Example. Your teammate clones the repo, runs `npm install`, and their server behaves differently from yours. The likely cause: `package-lock.json` was never committed, so their install resolved every `^` range again and picked newer versions. With the lockfile in the repository, `npm ci` on their machine installs the very same version of every package you have.

Common mistake. Deleting `package-lock.json` "to fix" an installation problem and committing the new one. That silently upgrades every transitive dependency at once. Delete `node_modules/` instead (it is disposable) and run `npm ci`; regenerate the lockfile only on purpose.

Layout and configuration

A Node/Express project layout

A course project keeps its code in `src/`, one folder per job: `routes/` (**URL → function**), `controllers/` (**what each request does**), `models/` (**database access**), `middleware/` (**steps shared by many requests**) and `config/` (**settings**). `app.js` **builds the app**; `server.js` **starts it**.

Folder tree

```
my-api/
├─ package.json
├─ package-lock.json
├─ .gitignore          # node_modules/ and .env
├─ .env                # real settings: NOT committed
├─ .env.example        # same keys, placeholder values: committed
├─ README.md
├─ src/
│  ├─ server.js        # requires app, calls app.listen()
│  ├─ app.js           # builds and exports the Express app
│  ├─ routes/          # method + URL → controller function
│  ├─ controllers/     # request logic: read req, send res
│  ├─ models/          # database queries (once there is a database)
│  ├─ middleware/      # logger, auth, error handler
│  └─ config/          # reads process.env in one place
└─ tests/              # automated tests (supertest)
```

Example. Follow `GET /tasks` through this layout: `server.js` started the app → `app.js` passed the request through `express.json()` and the logger middleware → `app.use('/tasks', tasksRouter)` handed it to `routes/tasks.js` → the router matched `GET /` and called `listTasks` from `controllers/tasksController.js` → the controller answered with `res.json(...)`. Five files, one small job each.

Common mistake. Calling `app.listen()` inside `app.js` "because it is shorter". It works until the first test file requires the app: every test run then opens a real port, and a second test file fails with `EADDRINUSE`. Keep the rule strict: `app.js` exports, `server.js` listens.

Environment variables and `process.env`

An environment variable is a named setting that the system starting a program hands to it, outside the code. Node exposes them in `process.env`, an object whose values are always strings, or undefined for a variable that was never set.

```
// In Node this object would be process.env; here we fake it.
const env = { PORT: '4000', DEBUG: 'false' };
```

```
const port = env.PORT || 3000;
console.log(port, typeof port);           // 4000 string

const portNumber = Number(env.PORT) || 3000;
console.log(portNumber, typeof portNumber); // 4000 number

if (env.DEBUG) console.log('debug is on?!'); // runs: 'false' is a non-empty string
console.log(env.DEBUG === 'true');           // false: compare the text

console.log(env.DATABASE_URL);              // undefined: never set
```

Example. Port 3000 is already used on your teammate's laptop. Instead of editing `server.js` (and fighting over it in every merge), they start the API with `PORT=4000 npm start` in Git Bash, or `$env:PORT = 4000; npm start` in PowerShell. `process.env.PORT` is `'4000'`, the server listens on 4000, and the code in Git never changed.

Common mistake. Testing a flag with `if (process.env.DEBUG)` when it is set to `false`. The value is the **string** `'false'`, which is **truthy** (every non-empty string is), so debug mode switches on. Compare explicitly: `process.env.DEBUG === 'true'`. The same trap with numbers: `process.env.PORT + 1` is `'40001'`, not `4001`.

.env files, config and keeping secrets out of Git

A **.env file** lists a project's environment variables (`NAME=value`, one per line) so you do not type them every time. Because it holds secrets it is **git-ignored**; the repository gets a `.env.example` with the same names and placeholder values, and one config module reads every variable.

```
// .env (git-ignored)           // .env.example (committed)
// PORT=3000                    // PORT=3000
// DATABASE_URL=postgres://ana:s3cret@localhost:5432/tasks
//                               // DATABASE_URL=postgres://user:password@localhost:5432/tasks
// JWT_SECRET=k8Jq2v...        // JWT_SECRET=change-me

// src/config/index.js: the only file that reads process.env
function required(name) {
  const value = process.env[name];
  if (!value) throw new Error(`Missing environment variable ${name} (see .env.example)`);
  return value;
}

module.exports = {
  port: Number(process.env.PORT) || 3000,
  databaseUrl: required('DATABASE_URL'),
  jwtSecret: required('JWT_SECRET'),
};

// src/server.js: load .env FIRST, before anything reads the config
require('dotenv').config();           // or start with: node --env-file=.env src/server.js
const config = require('./config');
const app = require('./app');
app.listen(config.port, () => console.log(`Listening on http://localhost:${config.port}`));
```

Example. A new teammate clones the repo, runs `npm ci` and `npm start`, and immediately gets `Error: Missing environment variable DATABASE_URL (see .env.example)`. They copy the example file (`cp .env.example .env` in Git Bash, Copy-Item `.env.example .env` in PowerShell), fill in their own database password and start again. Without fail-fast, the server would have started "fine" and crashed on the first request that touched the database, far from the real cause.

Common mistake. Loading `.env` too late. If `server.js` requires `./config` (or `./app`, which requires the config) **before** `require('dotenv').config()`, the config module runs first, finds no variables, and throws "missing variable" although the value is right there in `.env`. Load `.env` on the first line of the entry file, or use `--env-file`. And never "fix" a missing variable by pasting the real secret into the code as a default.

Asynchronous code

Blocking and non-blocking: why Node code is asynchronous

A **synchronous** (blocking) call makes the program wait until it has finished. An **asynchronous** (non-blocking) call starts the work, returns at once, and runs a function you supply **later**, when the result is ready. Node uses asynchronous calls for everything slow (disk, network, database), so one thread can serve many requests.

```
const fs = require('node:fs');

// Blocking: the next line waits until the whole file has been read
const text = fs.readFileSync('notes.txt', 'utf8');
console.log('1. read', text.length, 'characters');

// Non-blocking: start reading, hand over a callback, carry on
fs.readFile('notes.txt', 'utf8', (err, data) => {
  console.log('3. the callback runs when the file has arrived');
});
console.log('2. this line runs before the file arrives');
```

Example. Two users call your API at the same moment. Handler A runs a database query that takes 80 ms; handler B just answers { status: 'ok' }. With an asynchronous query, Node starts A's query, answers B at once, and finishes A when the database replies: B waits 0 ms. With a blocking query, B would wait the full 80 ms behind A, and with 100 users the last one would wait 8 seconds.

Common mistake. Expecting an asynchronous result on the next line: `let data; fs.readFile('notes.txt', 'utf8', (err, d) => { data = d; }); console.log(data);` prints undefined, because the `console.log` runs **before** the callback does. Use the value **inside** the callback, or, better, with `await` (two cards ahead).

Callbacks and the error-first convention

The original Node style: you pass a **callback** as the last argument, and Node calls it with `(err, result)` when the work is done. `err` is null on success and an `Error` object on failure, so every callback starts by checking it.

```
const fs = require('node:fs');

// Three steps that depend on each other: "callback hell"
fs.readFile('config.json', 'utf8', (err, text) => {
  if (err) return console.error(err);
  const config = JSON.parse(text);
  fs.readFile(config.dataFile, 'utf8', (err, data) => {
    if (err) return console.error(err);
    fs.writeFile('copy.txt', data, (err) => {
      if (err) return console.error(err);
      console.log('done');
    });
  });
});

// A function written in Node's callback style (this one answers at once)
function divide(a, b, callback) {
  if (b === 0) {
    callback(new Error('Cannot divide by zero')); // failure: the error first
    return;
  }
  callback(null, a / b); // success: null, then the result
}

divide(10, 2, (err, result) => {
  if (err) return console.log('Error:', err.message);
  console.log('Result:', result); // Result: 5
});

divide(1, 0, (err, result) => {
  if (err) return console.log('Error:', err.message); // Error: Cannot divide by zero
});
```

```
console.log('Result:', result);
});
```

Example. `fs.readFile('missing.txt', 'utf8', (err, data) => { ... })`: the file does not exist, so Node calls the callback with an Error whose code is `'ENOENT'` ("no such file or directory"), and data is undefined. The line `if (err) return console.error(err);` reports it and stops; without it, the next line would use the missing data and crash.

Common mistake. Leaving out the `return` in `if (err) callback(err);`. The function reports the error **and then carries on** with the success path, so the callback is called twice or works with a result that does not exist. Write `if (err) return callback(err);`. Also: `try { fs.readFile(..., cb); } catch (e) { ... }` does not catch the file error, because the error happens later, after the `try` block has already finished. Callback errors arrive in `err`.

Promises: a value that arrives later

A **promise** is an object that stands for a result that is not ready yet. It starts **pending** and settles once: **fulfilled** with a value or **rejected** with an error. `.then(fn)` runs `fn` with the value, `.catch(fn)` with the error, and the calls chain instead of nesting.

```
const fs = require('node:fs/promises');

// The three dependent steps of the callback card, flat
fs.readFile('config.json', 'utf8')
  .then((text) => JSON.parse(text)) // a value → passed on
  .then((config) => fs.readFile(config.dataFile, 'utf8')) // a promise → waited for
  .then((data) => fs.writeFile('copy.txt', data))
  .then(() => console.log('done'))
  .catch((err) => console.error('Something failed:', err.message)); // any step

// Making your own promise: wait ms milliseconds
function sleep(ms) {
  return new Promise((resolve) => setTimeout(resolve, ms));
}

sleep(500).then(() => console.log('half a second later'));
```

Example. `fetch('http://localhost:3000/tasks/1')` returns a pending promise at once. `.then((res) => res.json())` runs when the response arrives and returns another promise (reading the JSON body); the next `.then((task) => console.log(task.title))` receives the parsed object. If the server is not running, the first promise is **rejected**: none of the `.then` callbacks run, and the `.catch` at the end prints the error.

Common mistake. Starting more async work inside a `.then` without returning it: `.then((config) => { fs.readFile(config.dataFile, 'utf8'); })`. With braces and no `return`, the callback returns undefined, so the next `.then` runs immediately instead of waiting for the file, and a failure of that read escapes the final `.catch`. Write `return fs.readFile(...)`, or use the short arrow form without braces.

async and await

An `async` function **always returns a promise**. Inside it, `await` promise **pauses that function until the promise settles, then gives back its value (or throws its error)**. The code reads top to bottom like synchronous code, yet the thread is never blocked.

```
const fs = require('node:fs/promises');

async function copyData() {
  const text = await fs.readFile('config.json', 'utf8'); // pauses here; the thread stays free
  const config = JSON.parse(text);
  const data = await fs.readFile(config.dataFile, 'utf8');
  await fs.writeFile('copy.txt', data);
  return data.length; // fulfils the returned promise
}

copyData().then((n) => console.log(`copied ${n} characters`));
console.log('this prints first: copyData() returned a pending promise');
```

```
// The same shape as copyData, with a fake slow read instead of the file system
const sleep = (ms) => new Promise((resolve) => setTimeout(resolve, ms));
async function readSlowly(name) {
  await sleep(200); // pretend this is a disk read
  return `contents of ${name}`;
}

async function copyData() {
  console.log('copyData starts');
  const text = await readSlowly('config.json'); // pauses only copyData
  console.log('got:', text);
  return text.length;
}

copyData().then((n) => console.log('copied', n, 'characters'));
console.log('this prints second: copyData() returned a pending promise');
```

Example. The fetch from the promises card, with await: `async function showTask() { const res = await fetch('http://localhost:3000/tasks/1'); const task = await res.json(); console.log(task.title); }`. One await per promise, in the order they are needed. An Express handler has the same shape: `app.get('/tasks', async (req, res, next) => { ... await ... })`.

Common mistake. Forgetting await: `const task = db.findTask(id); console.log(task.title);` prints undefined, because task is a **promise**, not the task, and a promise has no title. In a route, `res.json(db.findAll())` sends {}, the JSON of a promise object. When a value prints as `Promise { <pending> }` or arrives as {}, look for the missing await.

Errors in async code: try/catch every await

When an awaited promise is rejected, await **throws** the error on that line, so you catch it with an ordinary `try { ... } catch (err) { ... }`. In an Express 4 route the catch must pass the error on with `next(err)`: an error nobody catches stops the whole server.

```
// Express 4: an error here ends the whole server (unhandled rejection)
app.get('/tasks', async (req, res) => {
  const rows = await db.tasks.findAll(); // rejects if the database is down
  res.json(rows);
});

// Safe pattern: catch, and forward to the error-handling middleware
app.get('/tasks', async (req, res, next) => {
  try {
    const rows = await db.tasks.findAll();
    res.json(rows);
  } catch (err) {
    next(err); // → app.use((err, req, res, next) => { ... })
  }
});
```

Example. The database is restarting when a user calls GET /tasks. With the first handler, the terminal prints `Error: connect ECONNREFUSED` with a stack trace and the Node process **exits**: that request never gets an answer, and every other request fails until someone restarts the server (`--watch` restarts on a file change, not after a crash). With the second handler, `next(err)` reaches the error middleware, this one user gets a 500 with `{ "error": "..." }`, and the server keeps serving everyone else.

Common mistake. "The error middleware will catch it anyway." In Express 4 it only receives errors **thrown synchronously** in a handler or passed with `next(err)`; a throw after an await happens later, out of Express's reach. A second trap: `try { saveTask(task); } catch (err) { ... }` **without** await. The call returns a promise immediately, the try block ends, and the rejection arrives after the net has been taken down. The await must be **inside** the try.

Independent work in parallel: Promise.all

Awaiting slow operations one after another runs them **in sequence** and adds up their times. When they do not depend on each other, start them all and wait once: `await Promise.all([a(), b()])` takes roughly as long as the slowest one.

```
async function loadDashboard(id) {
  // Sequential: about 100 ms + 120 ms = 220 ms
  // const user = await db.users.findById(id);
  // const tasks = await db.tasks.findByOwner(id);

  // Parallel: both start now; about max(100, 120) = 120 ms
  const [user, tasks] = await Promise.all([
    db.users.findById(id),
    db.tasks.findByOwner(id),
  ]);
  return { user, tasks };
}

// Fake queries: each answers after ms milliseconds
const sleep = (ms, value) => new Promise((resolve) => setTimeout(() => resolve(value), ms));
const findUser = () => sleep(100, { name: 'Ana' });
const findTasks = () => sleep(120, ['Buy milk', 'Study']);

async function sequential() {
  const t0 = Date.now();
  const user = await findUser();
  const tasks = await findTasks();
  console.log('sequential:', Date.now() - t0, 'ms', user.name, tasks.length);
}

async function parallel() {
  const t0 = Date.now();
  const [user, tasks] = await Promise.all([findUser(), findTasks()]);
  console.log('parallel:', Date.now() - t0, 'ms', user.name, tasks.length);
}

sequential().then(parallel);
```

Example. A dashboard route needs the user, their tasks and their unread notifications: three queries of about 50 ms that each need only the user id from the URL. Awaiting them one by one answers in about 150 ms; `const [user, tasks, notes] = await Promise.all([...])` answers in about 50 ms, three times faster, still inside one try/catch.

Common mistake. Writing `await` inside the array: `Promise.all([await a(), await b()])`. The awaits run one after the other **while the array is being built**, so the work is sequential again and `Promise.all` receives plain values. Pass the promises themselves: `Promise.all([a(), b()])`. Also remember that results come back in the order of the array, not in the order the operations finished.

The event loop

The event loop: one thread, a call stack and queues

The **event loop** is how a single JavaScript thread handles many things: code runs on the **call stack** until it is empty; finished timers and I/O put their callbacks in the **task queue**; promise callbacks go to the **microtask queue**; and the loop keeps taking the next callback and running it, one at a time.

- 1. Run the script until the call stack is empty.
- 2. Run every microtask, until the microtask queue is empty.
- 3. Take one task from the task queue (an expired timer, a finished I/O operation) and run it to the end.
- 4. Back to step 2. When nothing is left to run or wait for, a Node program exits.

```
console.log('script start');
setTimeout(() => console.log('timeout'), 0);
Promise.resolve().then(() => console.log('promise'));
```

```
console.log('script end');
// Predict the four lines, then press Run.
```

Example. In the code above, both plain `console.log` lines run while the script is on the stack. `setTimeout` hands its callback to the timer; after 0 ms it goes to the task queue. `.then` on an already-fulfilled promise puts its callback straight into the microtask queue. When the script ends, the loop empties the microtask queue first (promise) and then takes the task (timeout). Step through it below with the "Promise.then versus setTimeout" program and watch each callback move.

Common mistake. Reading `setTimeout(fn, 1000)` as "run `fn` in exactly one second". It means "put `fn` in the task queue **no sooner than** one second from now". If the stack is still busy then, or other tasks are ahead, it runs later. A timer delay is a minimum, not an appointment.

Microtasks before tasks: predicting the order

To predict what asynchronous code prints: first all synchronous code; then every microtask (promise callbacks, code after `await`, `queueMicrotask(fn)`) in the order they were queued; then tasks such as timers, one at a time, each followed by all the microtasks it created.

Where each kind of callback waits

You write	It waits in	It runs
<code>console.log(...)</code> , any plain function call	nothing: it runs now	Immediately, on the call stack
<code>.then(fn)</code> , <code>.catch(fn)</code> , code after <code>await</code> , <code>queueMicrotask(fn)</code>	Microtask queue	As soon as the stack is empty, before any task
<code>setTimeout(fn, ms)</code> , <code>setInterval(fn, ms)</code>	Task queue, once the delay has passed	One at a time, after all microtasks
I/O callbacks: <code>fs.readFile</code> , an incoming HTTP request	Task queue, once the I/O has finished	One at a time, after all microtasks
<code>process.nextTick(fn)</code> (Node only)	<code>nextTick</code> queue	Before the promise microtasks
<code>setImmediate(fn)</code> (Node only)	Check queue	In the check phase of each loop round

```
console.log('1');
setTimeout(() => {
  console.log('2');
  Promise.resolve().then(() => console.log('3'));
}, 0);
Promise.resolve().then(() => {
  console.log('4');
  setTimeout(() => console.log('5'), 0);
});
console.log('6');
// Predict the order, then press Run. Then swap two lines and predict again.
```

Example. Applying the rules to the code above: 1 and 6 are synchronous. The `.then` callback is a microtask, so 4 comes next, and it schedules a second timer. Now the first timer's task runs and prints 2; it queues a microtask, which runs before the next task: 3. Last comes the second timer: 5. Result: 1, 6, 4, 2, 3, 5.

Common mistake. Ordering callbacks by the line they are written on. `setTimeout` on line 2 and `.then` on line 6 say nothing about which runs first. What decides is **which queue** a callback joins and **when** it joins it. Ask two questions about every callback: which queue? When is it queued: now, when the timer expires, or when its promise settles?

Never block the event loop

Blocking the event loop means keeping the call stack busy with long synchronous work. Meanwhile no callback can run: timers fire late and, in a server, **every** other request waits. Await slow I/O, and move heavy computation off the main thread.

```
// One slow route freezes the whole server
app.get('/slow', (req, res) => {
  const end = Date.now() + 5000;
  while (Date.now() < end) {} // 5 s of busy waiting on the call stack
```

```
res.json({ done: true });
});

app.get('/health', (req, res) => res.json({ status: 'ok' }));
// While /slow runs, GET /health also waits up to 5 s: it is stuck in the queue.
```

Example. Start the server, run `curl http://localhost:3000/slow` in one terminal and straight away `curl http://localhost:3000/health` in another. `/health` normally answers in a few milliseconds, but now it waits until the five seconds are over: its request has arrived, but the event loop cannot take it while `/slow` occupies the stack. The visualiser's "A blocking loop delays a timer" program shows the same effect with a timer.

Common mistake. Thinking `async` makes a function non-blocking. `async function work() { for (let i = 0; i < 1e9; i++) {} }` still blocks for the whole loop: `async` only changes what the function **returns** (a promise). Code stops blocking only where it really waits for something outside the thread (I/O, a timer, a worker), at an `await`.